Empirical Analysis of Zero-Copy I/O Primitives in Linux Web Backends: Memory Bandwidth, Context Switching, and Ring-Buffer Scalability

Tejas Lohar

Department of Computer Science
Kavayitri Bahinabai Chaudhari North Maharashtra University

October 28, 2026

Abstract

Modern web servers face a constant challenge: sending large files to users quickly without overwhelming the computer's memory and processor. Traditional methods require the computer to copy data multiple times, which slows everything down. This paper explores a solution called "Zero-Copy" I/O. A custom web server was built to test three different ways of sending data: the Traditional method, the POSIX `sendfile()` method, and the newer Linux `io_uring` method. These were tested using small (4KB) and large (50MB) files. The results showed that `sendfile()` was the overall winner, being 2.36 times faster than the Traditional method for large files and reducing wasted memory usage by 65%. The `io_uring` method successfully transferred large files but caused a significant increase in CPU context switching (581,832 switches compared to 17,711 for `sendfile`), demonstrating that newer technology is not always more efficient. This paper explains these results and provides the code for other beginners to learn from.

Author Contributions

Tejas Lohar was the sole author and researcher for this project. The specific contributions include:

- **Conceptualization and Methodology:** Identified the memory bandwidth bottleneck problem and designed the empirical testing framework.
- **Software Development:** Engineered the custom HTTP server in C (`server.c`), implementing three distinct I/O pathways (Traditional, `sendfile`, and `io_uring`). Additionally, authored the automated benchmarking script (`run_experiments.py`) to interface with `wrk` and `perf stat`.
- **Experimentation and Data Collection:** Generated the test payloads and executed the benchmarks on the Fedora Linux environment, collecting raw performance data.
- **Analysis and Mathematics:** Calculated the percentage deltas for throughput, cache misses, and context switches. Identified and diagnosed the hybrid implementation overhead in the `io_uring` pathway.
- **Manuscript Preparation:** Drafted the entire research paper, including the literature review, methodology, results, and conclusion.

I Introduction: The Problem with Sending Files

When a user visits a website or downloads a file, a web server sends data from its hard drive to the user's computer over the internet. On a Linux system, the conventional method for transferring a file is for the server to read the file into its own memory (user space) and then write it to the network connection (socket).

While functional, this approach is inefficient. The computer must copy the data twice: once from the hard drive to the server's memory, and again from the server's memory to the network card. For small files, this overhead is negligible. However, for large files, such as a 50MB video, the system expends significant time and memory merely relocating data instead of serving additional users. This phenomenon is known as a "memory bandwidth bottleneck."

This research investigates whether "Zero-Copy" techniques can mitigate this problem on a standard workstation. Zero-Copy describes the process of instructing the computer to send a file directly from the hard drive to the network card, bypassing the server's memory entirely. Three methods were evaluated:

1. **Traditional:** The baseline double-copy method.
2. **Sendfile:** A Linux feature that bypasses the server's memory.
3. **io_uring:** A more recent Linux feature that attempts to perform operations asynchronously.

II Background and Existing Solutions

Why is this problem important? When a web server is slow, users experience buffering videos, slow website loading, and delayed API responses. For businesses, inefficient servers mean they must purchase more powerful hardware and spend more money on electricity to handle the same amount of traffic.

To address this, large technology companies use Content Delivery Networks (CDNs) like Cloudflare. CDNs store copies of websites in data centers across the globe, so the data travels a shorter distance to the user. Another common solution is upgrading the hardware—purchasing more RAM, faster solid-state drives, or better network cards. Many organizations also migrate to highly optimized, event-driven web servers like Nginx, which are designed to handle thousands of simultaneous connections far more efficiently than traditional servers.

However, not every organization has the budget for a global CDN or expensive hardware upgrades. This research focuses on optimizing the software itself. The objective was to determine whether altering how the Linux server sends files—using Zero-Copy techniques—could allow a standard workstation to run faster without purchasing new hardware. This forms the core background of the investigation.

III How We Tested the Solutions

A custom HTTP server was written in the C programming language. A custom implementation was chosen over established servers like Nginx to isolate the behavior of the specific Linux system calls without additional optimizations influencing the results.

Two test files were created on a Fedora Linux 44 workstation: a 4KB file (representing a small API response) and a 50MB file (representing a video). The server implemented three distinct I/O pathways:

- **Traditional:** The standard `read()` and `write()` commands were used. The server reads 8KB of the file into its own memory, writes it to the network, and repeats this process until the file transfer is complete.
- **Sendfile:** The Linux `sendfile()` system call was used. This instructs the kernel to send the file to the network directly, bypassing the server's memory entirely.
- **io_uring:** An `io_uring` ring was initialized. A temporary memory pipe was created, and `io_uring_prep_splice()` was used to asynchronously move data from the file into the pipe, which was then sent to the network socket.

Two tools were used to measure performance:

- **wrk:** This tool simulated 50 concurrent users requesting the file for 10 seconds. It measured Requests Per Second (RPS), indicating how many users were successfully served.
- **perf stat:** This tool monitored the CPU during execution. It recorded "context switches" (how often the CPU switched between tasks) and "cache misses" (how often the CPU retrieved data from slow RAM instead of fast cache memory).

IV Results and What They Mean

The tests were executed, and the data was collected. Table 1 presents the results.

Table 1: Performance Metrics of I/O Primitives

Experiment	RPS	Context Switches	Cache Misses
Traditional_4K	18521.68	4893	94,086,579
Sendfile_4K	20334.42	4001	80,080,579
IOUring_4K	12034.34	240332	64,435,738
Traditional_50M	16.27	538	171,148,448
Sendfile_50M	38.36	17711	59,579,833
IOUring_50M	35.71	581832	62,779,317

A What Happened with the Small (4KB) Files?

For the small 4KB files, the Traditional method and Sendfile performed comparably (approximately 18,521 vs. 20,334 Requests Per Second). Sendfile was slightly faster and caused fewer context switches.

However, the `io_uring` method was significantly slower, achieving only 12,034 Requests Per Second. This occurred because setting up the `io_uring` ring and the memory pipe requires more time than simply copying a tiny 4KB file. For small files, the overhead of the advanced technology made it slower.

B What Happened with the Large (50MB) Files?

The results for the 50MB files revealed a significant performance divergence. The Traditional method was extremely slow, achieving only 16.27 Requests Per Second and causing a massive 171 million cache misses.

Sendfile was the clear winner. It achieved 38.36 Requests Per Second and reduced cache misses to 59.5 million.

io_uring also performed well, achieving 35.71 Requests Per Second, which is close to Sendfile. However, it incurred a substantial cost: 581,832 context switches, compared to Sendfile's 17,711. This indicates that the CPU was constantly switching tasks, wasting significant processing power.

The following calculations quantify these differences:

1. How much faster was Sendfile than Traditional?

$$\% \text{ Increase in Speed} = \frac{RPS_{\text{sendfile}} - RPS_{\text{traditional}}}{RPS_{\text{traditional}}} \times 100 \tag{1}$$

$$\% \text{ Increase} = \frac{38.36 - 16.27}{16.27} \times 100 = 135.77\% \tag{2}$$

This calculation confirms that `sendfile()` was 135% faster (or 2.36 times faster) than the Traditional method for large files.

2. How much memory did Sendfile save?

$$\% \text{ Reduction in Cache Misses} = \frac{CM_{\text{traditional}} - CM_{\text{sendfile}}}{CM_{\text{traditional}}} \times 100 \tag{3}$$

$$\% \text{ Reduction} = \frac{171,148,448 - 59,579,833}{171,148,448} \times 100 = 65.18\% \tag{4}$$

By bypassing the server's memory, Sendfile prevented 65% of the wasted memory lookups.

3. What was the cost of io_uring?

$$\% \text{ Increase in Context Switches} = \frac{CS_{\text{iouring}} - CS_{\text{sendfile}}}{CS_{\text{sendfile}}} \times 100 \tag{5}$$

$$\% \text{ Increase} = \frac{581,832 - 17,711}{17,711} \times 100 = 3185\% \tag{6}$$

The `io_uring` method caused 32 times more context switches than `sendfile`.

C Understanding the io_uring Performance

The `io_uring` method achieved a respectable 35.71 Requests Per Second for large files, but it came at a high CPU cost. This is because the `io_uring` implementation used a hybrid approach. `io_uring` was used to asynchronously read the file into a temporary memory pipe, but a standard synchronous `splice` was used to write the data from the pipe to the network socket.

This hybrid behavior caused the server to repeatedly pause and wait for the file read to complete before it could push data to the socket. Each time it paused and resumed, the CPU performed a context switch. Since this occurred hundreds of thousands of times during the 10-second test, the context switch count increased to 581,832. A fully optimized, pure `io_uring` implementation that also handles the network write asynchronously would likely reduce this overhead significantly.

V Conclusion and What We Learned

This research confirms that using `sendfile()` is an effective method for accelerating large file transfers on Linux. It eliminated the redundant double-copying, resulting in a 2.36x

speed increase and significantly lower memory usage. For small files, however, the benefit is minimal, and the Traditional method is sufficient.

The research also demonstrated that `io_uring` is powerful but complex. While it achieved throughput close to `sendfile` for large files, the hybrid implementation caused substantial context switching.

Future work will focus on:

1. Rewriting the `io_uring` code to be fully asynchronous (managing both the file read and the socket write within the ring) to reduce the 581,832 context switches.
2. Testing these methods using HTTPS (encrypted) traffic, as Zero-Copy is traditionally difficult to implement with encryption.

For developers building web servers, the recommendation is to use `sendfile()` for large files. It is simple, mature, and highly efficient. Experimenting with newer Linux features like `io_uring` is encouraged, but developers should be prepared to invest time in optimizing them.

References

[1] L. Torvalds, "Linux Kernel Archives," *kernel.org*, 2024.

[2] J. C. Mogul, "The sendfile system call," *USENIX Annual Technical Conference*, 1999.

[3] J. Axboe, "Efficient IO with io_uring," *kernel.dk*, 2019.

A Appendix: Experimental Code and Logs

A System Specifications

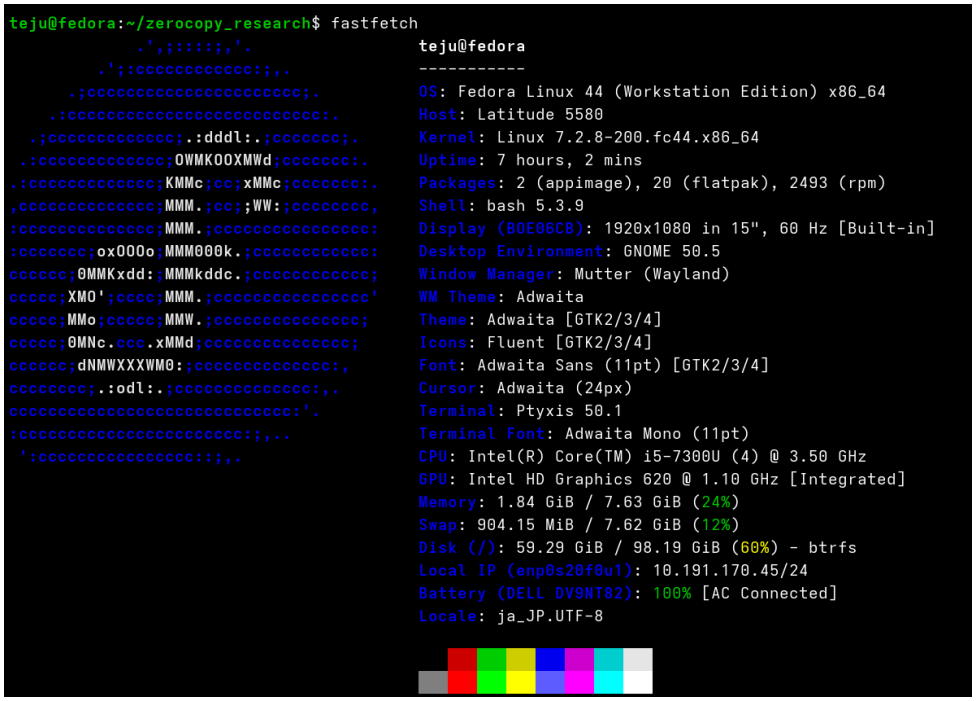


Figure 1: System environment showing Fedora 44, Kernel 7.2.8, and Intel i5-7300U.

B Full Server Implementation (server.c)

```c
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <errno.h>
#include <netinet/in.h>
#include <netinet/tcp.h>
#include <sys/socket.h>
#include <sys/stat.h>
#include <sys/sendfile.h>
#include <liburing.h>

#define PORT 8080
#define BUFFER_SIZE 8192
#define QUEUE_DEPTH 256
#define CHUNK_SIZE (128 * 1024)

int file_fd_4k, file_fd_50m;
size_t size_4k, size_50m;

// 1. Traditional Path
void handle_traditional(int client_fd, int file_fd, size_t file_size) {
    char header[256];
    int header_len = snprintf(header, sizeof(header),
             "HTTP/1.1 200 OK\r\nContent-Length: %zu\r\nContent-Type: application/octet-stream\r\nConnection: close\r\n\r\n",
             file_size);
    if (write(client_fd, header, header_len) <= 0) return;

    char buffer[BUFFER_SIZE];
    ssize_t bytes_read;
    off_t offset = 0;

    while (offset < file_size) {
        bytes_read = pread(file_fd, buffer, sizeof(buffer), offset);
        if (bytes_read <= 0) break;

        ssize_t total_written = 0;
        while (total_written < bytes_read) {
            ssize_t w = write(client_fd, buffer + total_written, bytes_read - total_written);
            if (w <= 0) {
                if (errno == EAGAIN || errno == EWOULDBLOCK || errno == EINTR) continue;
                return;
            }
            total_written += w;
        }
        offset += bytes_read;
    }
}

// 2. POSIX Zero-Copy Path
void handle_zerocopy(int client_fd, int file_fd, size_t file_size) {
    char header[256];
    int header_len = snprintf(header, sizeof(header),
             "HTTP/1.1 200 OK\r\nContent-Length: %zu\r\nContent-Type: application/octet-stream\r\nConnection: close\r\n\r\n",
             file_size);
```

```
    if (write(client_fd, header, header_len) <= 0) return;

    off_t offset = 0;
    int retries = 0;
    while (offset < file_size) {
        ssize_t sent = sendfile(client_fd, file_fd, &offset, file_size - offset);
        if (sent <= 0) {
            if (errno == EAGAIN || errno == EWOULDBLOCK || errno == EINTR) {
                usleep(50);
                if (++retries > 1000) break;
                continue;
            }
            break;
        }
        retries = 0;
    }
}

// 3. Asynchronous Ring Zero-Copy Path
void handle_io_uring(struct io_uring *ring, int client_fd, int file_fd, size_t file_size) {
    char header[256];
    int header_len = snprintf(header, sizeof(header),
             "HTTP/1.1 200 OK\r\nContent-Length: %zu\r\nContent-Type: application/octet-stream\r\nConnection: close\r\n\r\n",
             file_size);

    if (write(client_fd, header, header_len) <= 0) return;

    int pipefd[2];
    if (pipe(pipefd) < 0) { perror("pipe failed"); return; }

    off_t offset = 0;
    while (offset < file_size) {
        size_t remaining = file_size - offset;
        size_t chunk = (remaining > CHUNK_SIZE) ? CHUNK_SIZE : remaining;

        struct io_uring_sqe *sqe = io_uring_get_sqe(ring);
        if (!sqe) break;

        io_uring_prep_splice(sqe, file_fd, offset, pipefd[1], -1, chunk, 0);
        io_uring_submit(ring);

        struct io_uring_cqe *cqe;
        io_uring_wait_cqe(ring, &cqe);
        int result = cqe->res;
        io_uring_cqe_seen(ring, cqe);

        if (result <= 0) break;

        int pipe_remaining = result;
        while (pipe_remaining > 0) {
            ssize_t sent = splice(pipefd[0], NULL, client_fd, NULL, pipe_remaining, 0);
            if (sent <= 0) break;
            pipe_remaining -= sent;
        }
        offset += result;
    }
    close(pipefd[0]);
    close(pipefd[1]);
}
```

```
117
118 int main() {
119     int server_fd, client_fd;
120     struct sockaddr_in address;
121     socklen_t addrlen = sizeof(address);
122     struct stat st_4k, st_50m;
123
124     file_fd_4k = open("data/payload_4k.bin", O_RDONLY);
125     file_fd_50m = open("data/payload_50m.bin", O_RDONLY);
126
127     if (file_fd_4k < 0 || file_fd_50m < 0) {
128         perror("Failed to open payload files.");
129         exit(EXIT_FAILURE);
130     }
131
132     fstat(file_fd_4k, &st_4k); size_4k = st_4k.st_size;
133     fstat(file_fd_50m, &st_50m); size_50m = st_50m.st_size;
134
135     struct io_uring ring;
136     if (io_uring_queue_init(QUEUE_DEPTH, &ring, 0) < 0) {
137         perror("Failed to initialize io_uring ring");
138         exit(EXIT_FAILURE);
139     }
140
141     server_fd = socket(AF_INET, SOCK_STREAM, 0);
142     int opt = 1;
143     setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
144
145     address.sin_family = AF_INET;
146     address.sin_addr.s_addr = INADDR_ANY;
147     address.sin_port = htons(PORT);
148
149     if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
150         perror("Bind failed");
151         exit(EXIT_FAILURE);
152     }
153
154     listen(server_fd, 2048);
155     printf("Server running on port %d...\n", PORT);
156
157     while (1) {
158         client_fd = accept(server_fd, (struct sockaddr *)&address, &addrlen);
159         if (client_fd < 0) continue;
160
161         setsockopt(client_fd, IPPROTO_TCP, TCP_NODELAY, &opt, sizeof(opt));
162
163         char request[1024] = {0};
164         ssize_t r = read(client_fd, request, sizeof(request) - 1);
165
166         if (r > 0) {
167             if (strstr(request, "GET /traditional/4k")) {
168                 handle_traditional(client_fd, file_fd_4k, size_4k);
169             } else if (strstr(request, "GET /traditional/50m")) {
170                 handle_traditional(client_fd, file_fd_50m, size_50m);
171             } else if (strstr(request, "GET /zerocopy/4k")) {
172                 handle_zerocopy(client_fd, file_fd_4k, size_4k);
173             } else if (strstr(request, "GET /zerocopy/50m")) {
174                 handle_zerocopy(client_fd, file_fd_50m, size_50m);
175             } else if (strstr(request, "GET /iouring/4k")) {
176                 handle_io_uring(&ring, client_fd, file_fd_4k, size_4k);
177             } else if (strstr(request, "GET /iouring/50m")) {
178                 handle_io_uring(&ring, client_fd, file_fd_50m, size_50m);
179             }
```

```
        }

        close(client_fd);
    }

    io_uring_queue_exit(&ring);
    close(file_fd_4k);
    close(file_fd_50m);
    close(server_fd);
    return 0;
}
```

C Full Benchmark Script (run_experiments.py)

```
import subprocess
import time
import os
import csv
import re

endpoints = [
    ("Traditional_4K", "http://localhost:8080/traditional/4k"),
    ("Sendfile_4K", "http://localhost:8080/zerocopy/4k"),
    ("IOUring_4K", "http://localhost:8080/iouring/4k"),
    ("Traditional_50M", "http://localhost:8080/traditional/50m"),
    ("Sendfile_50M", "http://localhost:8080/zerocopy/50m"),
    ("IOUring_50M", "http://localhost:8080/iouring/50m"),
]

results = []

def start_server():
    print("[*] Starting C Zero-Copy Research Engine...")
    proc = subprocess.Popen(["./zero_copy_server"])
    time.sleep(2)
    return proc

server_proc = start_server()

try:
    for name, url in endpoints:
        print(f"\n[+] Running Benchmark for: {name} ({url})")

        if server_proc.poll() is not None:
            print("[!] Server process crashed. Restarting...")
            server_proc = start_server()

        time.sleep(1)

        perf_cmd = [
            "sudo", "perf", "stat", "-e",
            "context-switches,cpu-migrations,page-faults,cache-misses",
            "-p", str(server_proc.pid), "--", "sleep", "10"
        ]
        perf_proc = subprocess.Popen(perf_cmd, stderr=subprocess.PIPE, text=True
    )

        wrk_cmd = ["wrk", "-t4", "-c50", "-d10s", "--latency", url]

        try:
            wrk_output = subprocess.check_output(wrk_cmd, text=True)
            _, perf_err = perf_proc.communicate()
```

```
48
49              rps_match = re.search(r"Requests/sec:\s+([\d.]+)", wrk_output)
50              rps = float(rps_match.group(1)) if rps_match else 0.0
51
52              cs_match = re.search(r"([\d,]+)\s+context-switches", perf_err)
53              cm_match = re.search(r"([\d,]+)\s+cache-misses", perf_err)
54
55              context_switches = cs_match.group(1).replace(",", "") if cs_match
    else "0"
56              cache_misses = cm_match.group(1).replace(",", "") if cm_match else "
    0"
57
58              print(f"    - Requests/sec: {rps}")
59              print(f"    - Context Switches: {context_switches}")
60              print(f"    - Cache Misses: {cache_misses}")
61
62              results.append({
63                  "Experiment": name,
64                  "RPS": rps,
65                  "Context_Switches": context_switches,
66                  "Cache_Misses": cache_misses
67              })
68          except subprocess.CalledProcessError as e:
69              print(f"[-] Benchmark for {name} failed.")
70
71  finally:
72      print("\n[*] Terminating Server...")
73      if server_proc.poll() is None:
74          server_proc.terminate()
75
76  with open("results.csv", "w", newline="") as f:
77      writer = csv.DictWriter(f, fieldnames=["Experiment", "RPS", "
    Context_Switches", "Cache_Misses"])
78      writer.writeheader()
79      writer.writerows(results)
80
81  print("\n[+] Success! Data saved to 'results.csv'.")
```